ADDITIONAL INFO FOR JUDGES

Dock

A revision workspace, one lesson per space
github.com/omthorat2004/dock

Dock is a revision workspace scoped to a single lesson. A student creates a space for one lesson, shares the syllabus section it covers, and the lesson's topics are laid out as cards. Each card opens either a chat scoped to that one topic, or a shelf of YouTube explainers found for it.

Trying it out

> **Learn mode and the video shelf need your own Gemini API key.** Dock does not ship a shared key. After signing in, go to the API key page and paste a Google AI Studio key, then open any topic card. Without a key those two features answer 401 by design and everything else still works.

What to look at first	Create a space for one lesson with three or four topics, open the space, then open a topic card. The Videos button fills a shelf of five explainers; the Chat button opens learn mode for that topic alone.
Running it locally	Needs Python 3.12 or newer, Node, and MongoDB on port 27017. Both halves must run on `localhost` rather than `127.0.0.1`, because the auth cookies are SameSite lax and the two have to be same site.

```
# backend, from backend/
poetry install
poetry run fastapi dev src/app/main.py     # http://localhost:8000/docs

# frontend, from frontend/
npm install
npm run dev                                # http://localhost:3000
```

The backend needs a `.env`, documented key by key in `backend/.env.example`. Two matter for a full demo: `MONGODB_URI`, and `YOUTUBE_API_KEY` for a YouTube Data API v3 key, which is Dock's own server side key rather than the student's. Without it the video shelf returns 503 and says so in plain words instead of falling back to guesswork.

How it is built

Frontend

- Next.js 16 on the App Router, React 19
- Tailwind CSS v4
- TanStack Query for server state
- Zustand for auth state
- One axios instance, cookies only

Backend

- FastAPI on Python 3.14, Poetry src layout
- MongoDB via the async pymongo client
- PyJWT and bcrypt
- google-genai for the model calls
- YouTube Data API v3 for video search

Two architectural rules

Next.js never proxies the API. There are no route handlers, no server side fetching and no server actions for data. The browser calls FastAPI directly, so there is exactly one place a request is made and one place an

error is normalised.

The backend owns the session. It sets httpOnly cookies and the frontend stores nothing at all, so no token is ever reachable from JavaScript. Layering is strict on both sides: router to service to DAO on the backend, component to hook to api module to axios on the frontend.

The part worth a closer look

The first version of the video shelf asked the model to recommend explainers. It answered with well formed YouTube URLs whose ids belonged to no video at all, so every candidate was verified against YouTube and the dead ones dropped. That worked, and revealed the real problem: asking for five reliably left one or two. A model cannot know an eleven character video id from memory.

So the model no longer supplies ids, it supplies searches. It calls a `search_youtube` tool through a function calling loop, once for an Indian audience and once for a global one, and picks from what really came back. An id it writes itself selects nothing and disappears. The shelf then alternates between the two audiences, so a student gets both Indian teaching channels and international ones rather than whichever search happened to read better. If the model returns nothing usable, the searches already ran and the shelf is filled from them anyway.

What is built, and what is not

AREA	STATUS
Auth: register, sign in, sign out, refresh rotation with reuse detection	`Built`
Spaces: create for a lesson, list, open	`Built`
The space surface: pan, zoom, lesson centred with topic cards around it	`Built`
Learn mode: per topic chat, bounded by a rolling summary plus a ten message window	`Built`
Video shelf: tool driven YouTube search, five per request, twenty per topic	`Built`
Bring your own Gemini key, with model selection	`Built`
Topic extraction from the lesson text	`Not yet, typed by hand`
Card layout saved per space	`Not yet, derived`
Revision progress on a topic card	`Not yet, shows Not started`

Deliberately out of scope, and not planned: file uploads of any kind, assignments and grading, collaboration or shared spaces, and payments. Lessons and syllabus come in as text.

Testing and correctness

113 backend tests run against a real MongoDB database that is dropped between tests, with no mocking layer, because index behaviour and refresh rotation semantics are exactly what mocks get wrong. The suite covers auth and token rotation, hashing, spaces, the API key flow, provider error classification, chat limits and the video shelf, including the YouTube error paths.

```
cd backend && poetry run pytest -q          # 113 passed
cd backend && poetry run ruff check .
cd frontend && npx tsc --noEmit && npx eslint . && npm run build
```

Security notes

- Access tokens last 15 minutes, refresh tokens 30 days and are stored hashed with HMAC SHA256 so they can be rotated and revoked. Every refresh rotates the pair, and replaying an already rotated token is treated as a leak and ends every session for that user.
- Both tokens carry a type claim that is verified, so a refresh token cannot be presented as an access token.
- Passwords use bcrypt. Login failures return one generic message for every cause, so the API never reveals whether an email exists.
- The student's provider key is never returned to the client. The API only exposes whether one is set. It is stored as text because it has to be replayed to the vendor and therefore cannot be hashed; encrypting it at rest under a key separate from the session signing key is the next task on it, and is noted here rather than glossed over.

Dock. Source at github.com/omthorat2004/dock. Backend API docs are served at /docs on the API host.